

TECHNICAL REQUIREMENTS DOCUMENT (TRD)

Mehmoodah Academy — School Operating System (V1)

Internal School Platform

Version: 2.0

Status: Ready for Development

1. SYSTEM OVERVIEW

Mehmoodah Academy School Operating System is a web application designed to manage internal school operations.

Users:

- Admin
- Teachers
- Students

Scope:

- Attendance
- Student Records
- Results
- Homework
- Timetable
- Reports

Architecture: Frontend → Backend → Database → Storage

2. SYSTEM ARCHITECTURE

CLIENT

Flutter Web

↓

API LAYER

Business Logic



DATABASE

PostgreSQL



FILE STORAGE

Cloud Storage



MONITORING

Logs + Analytics

Architecture Type: Modular Monolith

Future: Multi-school ready

3. TECH STACK

Frontend: Flutter Web

State Management: Riverpod

Backend: Supabase

Database: PostgreSQL

Authentication: Supabase Auth

Storage: Supabase Storage

Hosting: Vercel

Notifications: Firebase

Version Control: GitHub

4. APPLICATION STRUCTURE

frontend/

lib/

core/

modules/

auth/

dashboard/

students/

teachers/

attendance/

results/

assignments/

timetable/

reports/

profile/

shared/

widgets/

services/

models/

routes/

backend/

database/

policies/

functions/

storage/

5. ROLE ACCESS CONTROL

ADMIN

Access: All Modules

Permissions: Create Read Update Delete

TEACHER

Access: Assigned Classes

Permissions: Read Update

Restricted: Delete

STUDENT

Access: Own Academic Records

Permissions: Read Only

6. DATABASE DESIGN

TABLE: users

id email role created_at

TABLE: teachers

id user_id name phone subject status

TABLE: students

id student_id name class_id roll_no guardian_name

TABLE: classes

id name section teacher_id

TABLE: attendance

id student_id class_id teacher_id date status

TABLE: exams

id name class_id term

TABLE: results

id exam_id student_id marks grade

TABLE: assignments

id teacher_id class_id title due_date

TABLE: submissions

id assignment_id student_id status

TABLE: announcements

id title body published_at

TABLE: timetable

id class_id teacher_id day

7. AUTH FLOW

User Login



Validate Credentials



Issue Session



Load Role



Redirect Dashboard

Security: JWT

Session: 7 Days

Refresh: Automatic

8. TEACHER CREATION FLOW

Admin



Open Teachers



Add Teacher



Create Auth User

↓

Insert Teacher Record

↓

Assign Role

↓

Activate Account

Validation: Unique Email

9. STUDENT CREATION FLOW

Admin

↓

Open Students

↓

Create Student

↓

Generate Student ID

↓

Assign Class

↓

Create Student Login

↓

Save

Validation: Unique Student ID

10. ATTENDANCE FLOW

Teacher



Select Class



Load Students



Mark Status



Submit



Store Records

Rules:

One attendance/day

Prevent duplicates

Statuses:

Present

Absent

Late

Leave

11. RESULT FLOW

Teacher



Create Exam



Input Marks



Calculate %



Generate Grade



Publish

Student:

Read Only

12. ASSIGNMENT FLOW

Teacher



Create Assignment



Upload File



Publish

Student



View



Submit

Teacher



Review

13. STORAGE

Store:

Photos

Homework

Documents

Reports

Structure:

students/

teachers/

assignments/

reports/

14. API ENDPOINTS

AUTH

POST /login

POST /logout

POST /reset

STUDENTS

GET /students

POST /students

PUT /students/{id}

DELETE /students/{id}

TEACHERS

GET /teachers

POST /teachers

PUT /teachers/{id}

ATTENDANCE

GET /attendance

POST /attendance

RESULTS

GET /results

POST /results

ASSIGNMENTS

GET /assignments

POST /assignments

REPORTS

GET /reports

15. PERFORMANCE

Dashboard: <2 sec

Attendance: <1 sec

Reports: <5 sec

Search: <500 ms

16. SECURITY

RBAC

HTTPS

Input Validation

Audit Logs

Rate Limits

Row Level Security

Password Hashing

17. ERROR HANDLING

400 Validation

401 Unauthorized

403 Forbidden

404 Missing

500 Server

UI: Readable Errors

18. DEPLOYMENT

DEV

↓

STAGING

↓

PRODUCTION

Pipeline:

GitHub

↓

Build

↓

Deploy

↓

Monitor

19. BACKUP

Database: Daily

Storage: Weekly

Retention: 30 Days

20. DEVELOPMENT ORDER

Phase 1 Auth

Phase 2 Students

Phase 3 Teachers

Phase 4 Attendance

Phase 5 Results

Phase 6 Assignments

Phase 7 Reports

Phase 8 Launch

END OF TRD